ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KĨ THUẬT MÁY TÍNH



CO5085 - Học sâu và ứng dụng trong thị giác máy tính

Báo cáo bài tập CO5085

Phân loại Fashion-MNIST bằng Softmax/MLP/CNN, Multi-Head Self-Attention và LSTM/GRU

E1/E2/E3

Giảng viên: Lê Thành Sách; Nguyễn Quốc Minh

No.	MSSV	Họ và tên
1	2570436	Tiêu Trí Kiệt

Thành phố Hồ Chí Minh, 2026

Contents

1 Dataset và splits **1**

2 E1 **1**

2.1 Kiến trúc 1

2.2 Huấn luyện 2

2.3 Bảng số liệu và kết quả 3

2.4 Đường cong học 4

2.5 Nhận xét 4

3 E2 **4**

3.1 Kiến trúc 4

3.2 Huấn luyện 6

3.3 Bảng số liệu và kết quả 6

3.4 Đường cong học 8

3.5 Nhận xét 9

4 E3 **9**

4.1 Kiến trúc 9

4.2 Huấn luyện 11

4.3 Bảng số liệu và kết quả 12

4.4 Đường cong học 14

4.5 Nhận xét 16

1 Dataset và splits

Trong các bài tập E1, E2, E3 đều sử dụng chung một tập dữ liệu là Fashion-MNIST, với cùng thiết lập, seed và cách chia tập dữ liệu.

Fashion-MNIST là tập dữ liệu chuẩn dùng trong học máy và thị giác máy tính, được tạo bởi Zalando Research vào năm 2017, với quy mô gồm 70.000 ảnh xám (grayscale) có cùng kích thước 28×28 pixel, gồm 10 lớp (classes) sản phẩm ở các nhóm quần/áo như áo phông/áo ba lỗ (T-shirt/top), quần dài (Trouser), áo len (Pullover), áo sơ mi (Shirt), váy (Dress), áo khoác (Coat), đến giày/dép như dép xăng-đan (Sandal), giày thể thao (Sneaker), bốt cổ thấp (Ankle boot) hay túi xách (Bag).

Bộ dữ liệu gốc gồm **60.000 ảnh train** và **10.000 ảnh test**, mỗi lớp có **1.000 ảnh test**.

Bảng 1. Các nhãn và lớp trong Fashion MNIST

Nhãn	Lớp	Nhãn	Lớp
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot

Fashion MNIST được thiết kế để đóng vai trò là bản thay thế "nặng ký" và thực tế hơn cho bộ dữ liệu chữ số viết tay MNIST. Đồng thời, có các hình dạng, chi tiết trang phục phong phú và phức tạp hơn, giúp kiểm tra hiệu năng (benchmark) tốt hơn cho các mô hình học sâu như Mạng nơ-ron tích chập (CNN), SVM, hay các thuật toán phân loại ảnh khác.

Trong các bài tập E1, E2, E3 sử dụng chung một thiết lập để chia tập dữ liệu thành các tập train, val và test như sau:

Bảng 2. Số ảnh và tỉ lệ của các tập dữ liệu

Tập	Số ảnh	Tỉ lệ
Train	48.000	80% của tập train gốc
Val	12.000	20% của tập train gốc
Test	10.000	-

2 E1

2.1 Kiến trúc

Cả ba mô hình đều trả về **logits**, không có lớp Softmax trong forward, vì trong `nn.CrossEntropyLoss` đã tự thực hiện log-softmax và negative log-likelihood:

$$\mathcal{L} = -\log \frac{e^{z_y}}{\sum_{j=1}^{10} e^{z_j}}$$

Softmax (hồi quy logistic đa lớp):

Ảnh ở input đầu vào có dạng `[B,C,H,W]` hay cụ thể là `[B, 1,28,28]`, trước khi đưa vào FC, ảnh cần được biến đổi thành một vector, có dạng `[B,C*H*W]` hay `[B,784]`.

Nói cách khác, ảnh được flatten thành $x \in \mathbb{R}^{784}$, sau đó đi qua một lớp tuyến tính $z = Wx + b$, với $W \in \mathbb{R}^{10\times784}$ và $b \in \mathbb{R}^{10}$.

Với xác suất dự đoán là: $P(y = i \mid x) = e^{z_i} / \sum_j e^{z_j}$ và chỉ được tính ở hàm `predict_proba`, tức là chỉ được tính khi suy luận.

Số tham số của softmax là: $784 \cdot 10 + 10 = 7.850$

MLP:

Mạng có hai lớp ẩn (hidden layer), tạo thành phép biến đổi: `784` $\rightarrow$ `512` $\rightarrow$ `256` $\rightarrow$ `10`. Sau mỗi hidden layer, có `ReLU` và `Dropout(0.2)`, tức là đều làm "tắt" đi khoảng 20% các neural để giảm bớt sự phụ thuộc các thông tin ở mạng.

Số tham số là: $401.920 + 131.328 + 2.570 = 535.818$

CNN:

Mạng CNN được thiết kế gồm 2 khối `Conv3x3` (với padding = 1) $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2`, với số kênh thay đổi là: $1 \rightarrow 32 \rightarrow 64$.

Kích thước không gian giảm theo $28 \rightarrow 14 \rightarrow 7$.

Head là `Flatten (64·7·7 = 3136)` $\rightarrow$ `Dropout(0.2)` $\rightarrow$ `Linear (3136` $\rightarrow$ `128)` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout(0.2)` $\rightarrow$ `Linear (128` $\rightarrow$ `10)`.

Số tham số là: $320 + 18.496 + 401.536 + 1.290 = 421.642$

2.2 Huấn luyện

Cả ba mô hình dùng chung một cấu hình, cũng như cách chạy vòng lặp huấn luyện, cụ thể như sau:

- Adam, learning rate (`lr=1e-3`), weight decay là 0, batch 128 và tối đa 50 epoch.
- Vơi `patience=5`, tức là kể từ khi chọn được giá trị tốt nhất (`best_epoch`) và sau đó 5 lần mà vẫn không tìm thấy được giá trị nào lớn hơn thì sẽ dừng, cũng là cơ chế **dừng sớm (early stopping)**.
- Theo dõi **val accuracy**, và seed 42.
- Thứ tự mini-batch giống nhau vì mỗi mô hình dùng một generator mới với cùng seed.

Vòng lặp huấn luyện:

1. Mỗi epoch, mô hình chuyển sang `train()`. Với mỗi batch, forward tính ra `logits`, tính `CrossEntropyLoss`, `optimizer.zero_grad()`, sau đó thực hiện `loss.backward()` để tính back prop và thực hiện update trọng số bằng `optimizer.step()`.

2. Train loss của epoch là trung bình có trọng số theo kích thước batch, train accuracy được tính bằng `correct/total` và cộng dồn trên mọi batch.

3. Sau đó đánh giá trên chế độ `val()`, tắt gradient (`torch.set_grad_enabled(False)` tương đương với `no_grad`), tính val loss và accuracy theo cùng cách.

4. Nếu val accuracy tăng thực sự, lưu bản sao trọng số và đặt `patience` về 0; ngược lại tăng lên 1 và dừng lại khi bộ đếm bằng 5.

5. Khi kết thúc, nạp lại trọng số của epoch có val accuracy cao nhất để tạo thành checkpoint, mọi số liệu train/val/test cuối cùng đều được tính trên checkpoint này.

2.3 Bảng số liệu và kết quả

Bảng 3. Kết quả đã chạy của E1

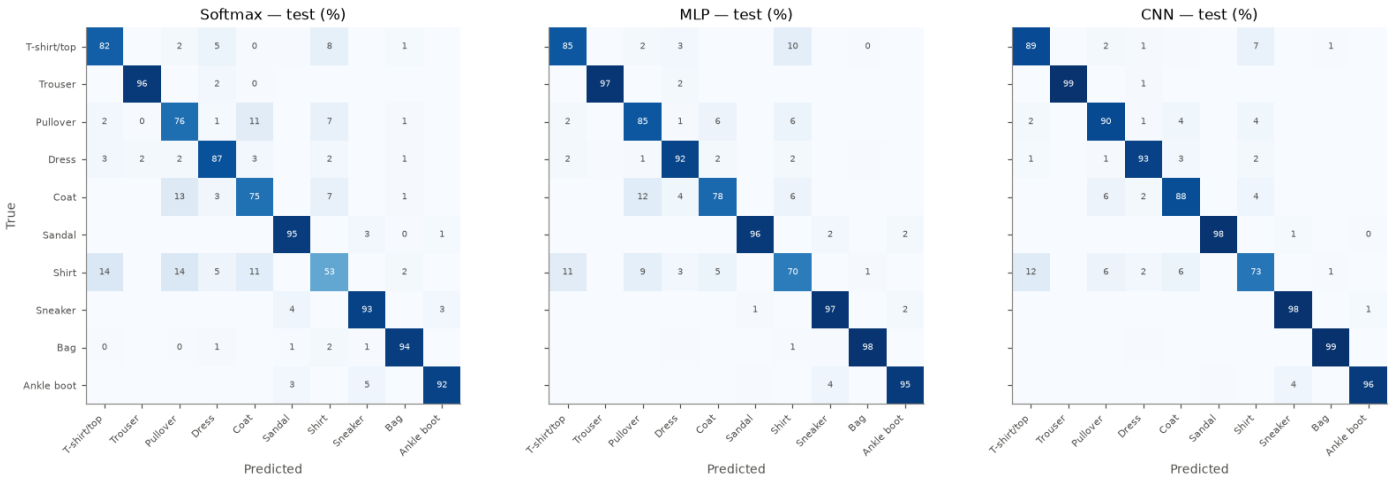
Mô hình	Tham số	Best epoch / số epoch đã chạy	Time (s)	Train (eval)	Val	Test	Số lỗi test
Softmax	7.850	14 / 19	2,9	87,03%	86,31%	84,41%	1.559
MLP	535.818	22 / 27	13,8	95,19%	90,26%	89,46%	1.054
CNN	421.642	21 / 26	217,1	98,49%	**93,27%**	**92,31%**	769

Bảng 4. Kết quả recall của E1

Lớp	Softmax	MLP	CNN
T-shirt/top	82	85	89
Trouser	96	97	99
Pullover	76	85	90
Dress	87	92	93
Coat	75	78	88
Sandal	95	96	98
Shirt	**53**	**70**	**73**
Sneaker	93	97	98
Bag	94	98	99
Ankle boot	92	95	96

Recall của lớp k theo định nghĩa là $\mathrm{TP}_k/(\mathrm{TP}_k + \mathrm{FN}_k)$, tức là tỉ lệ ảnh thuộc lớp k được dự đoán đúng.

Do Fashion-MNIST là tập dữ liệu cân bằng, nên trung bình recall bằng accuracy.

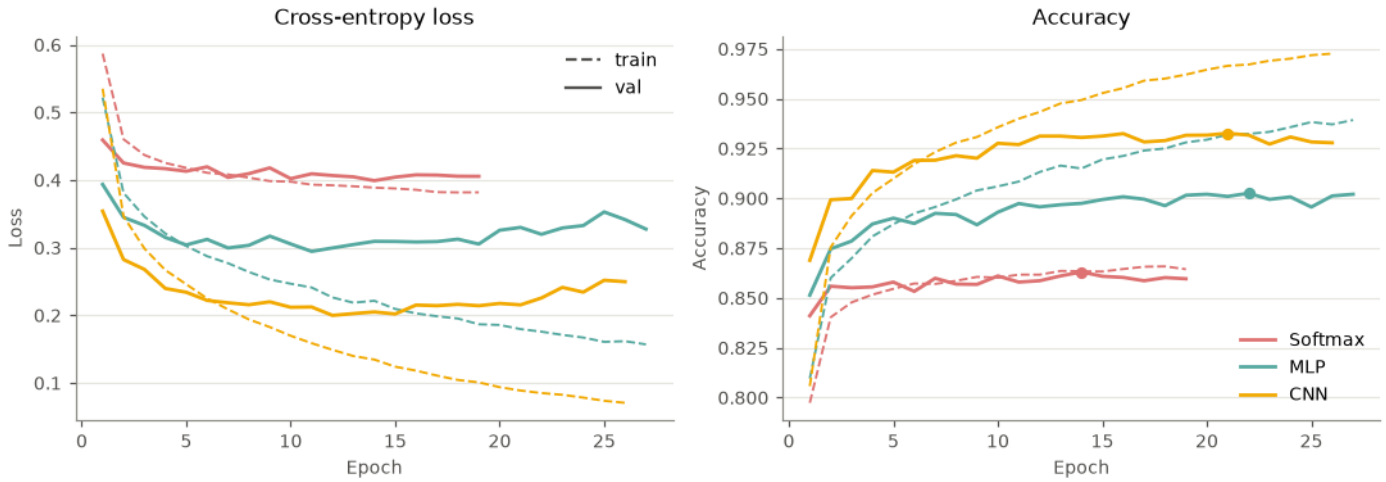


Hình 1. Confusion matrix chuẩn hóa theo hàng (%)

Qua đó, có thể rút ra được nhận xét đơn giản về các cặp nhầm lẫn chính theo thứ tự softmax, MLP và CNN là:

- Coat→Pullover: 13 / 12 / 6.
- Shirt→T-shirt: 14 / 11 / 12.
- T-shirt→Shirt: 8 / 10 / 7.
- Shirt→Pullover: 14 / 9 / 6.
- Shirt→Coat: 11 / 5 / 6.

2.4 Đường cong học



Hình 2. Cross-entropy loss (trái) và accuracy (phải). Nét đứt là train (đo trong lúc tối ưu, dropout bật), nét liền là val, chấm tròn là checkpoint được chọn. Màu đỏ là Softmax, xanh lá là MLP, vàng là CNN

Dựa vào đường cong học trên, có thể quan sát được các đặc điểm sau:

- **Softmax**: train và val gần như trùng nhau ngay từ vài epoch đầu. Mô hình bị underfit do thiếu capacity nên có huấn luyện lâu hơn cũng không giúp cải thiện.
- **MLP**: val loss thấp nhất là ≈0.29 ở epoch 11, sau đó tăng dần trong khi train loss tiếp tục giảm, đây là dấu hiệu của overfitting.
- **CNN**: val loss thấp nhất ≈0.2 ở epoch 12, sau đó train loss giảm xuống trong khi val loss tăng lên một chút, nên CNN cũng bị overfit.

2.5 Nhận xét

Dựa vào các kết quả thu được trong quá trình huấn luyện, thứ tự chất lượng rõ ràng là **CNN** > **MLP** > **Softmax** với test accuracy lần lượt là **92,31%** > **89,46%** > **84,41%**.

CNN dùng ít tham số hơn MLP, nhưng cao hơn về điểm test, điều này cho thấy sự phù hợp của mô hình với inductive bias về tính cục bộ và chia sẻ trọng số.

3 E2

3.1 Kiến trúc

Multi-head Self-attention (MSA) có input $X \in \mathbb{R}^{B\times N\times D}$, có h head và $d = D/h$:

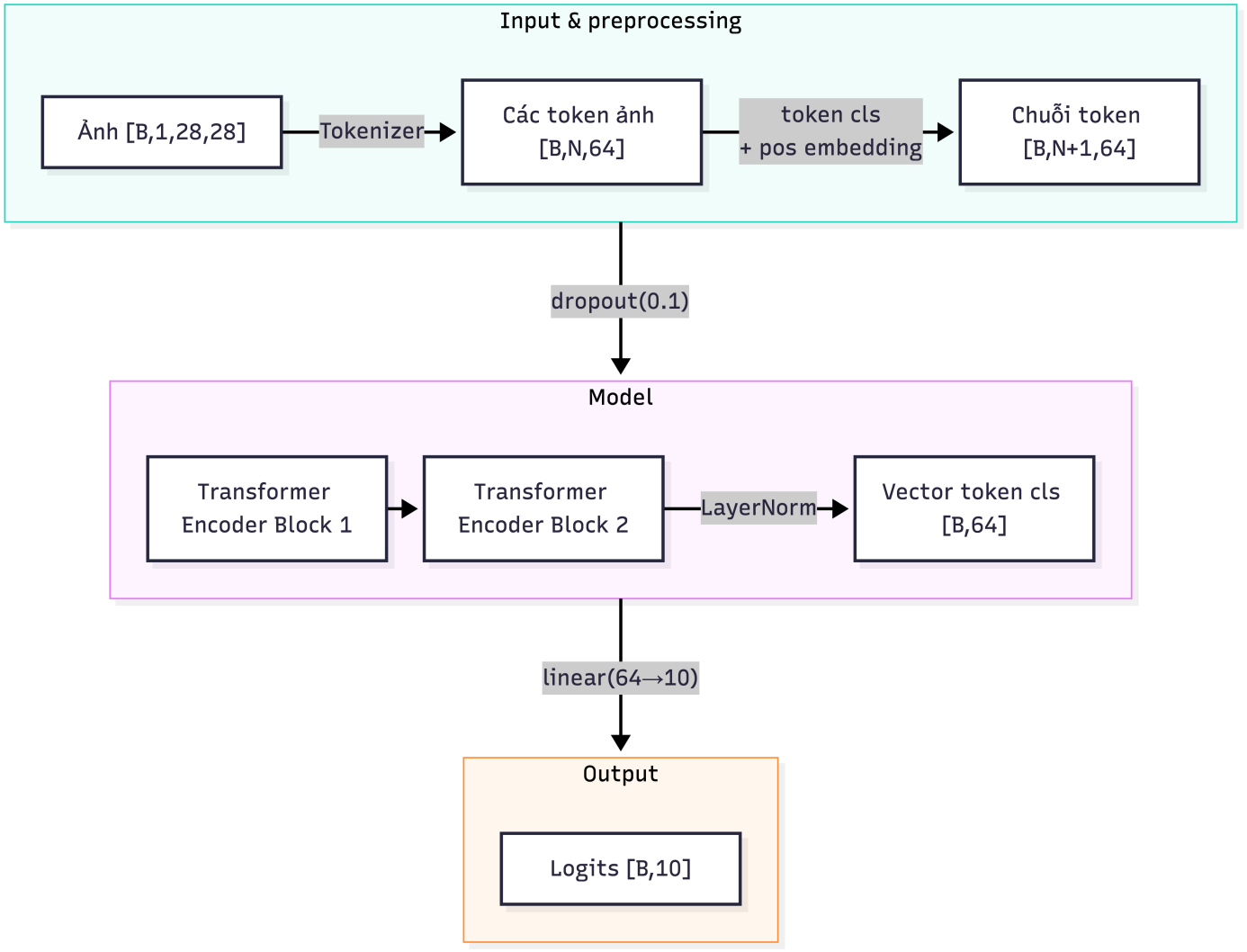
$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$
$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d}}\right) V_i$$
$$\text{MSA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O$$

Cụ thể, $Q, K, V \in \mathbb{R}^{B\times N\times d} \Rightarrow QK^\top \in \mathbb{R}^{B\times N\times N}$ hoặc $Q, K, V \in \mathbb{R}^{B\times h\times N\times d} \Rightarrow QK^\top \in \mathbb{R}^{B\times h\times N\times N}$, $\text{head}_i \in \mathbb{R}^{B\times N\times d}$.

Số lượng tham số của MSA là $4D^2 + 4D = 16.640$ với $D = 64$.

3.1.1 Transformer classifier

Là mô hình phân loại ảnh dựa trên mối quan hệ giữa các token của ảnh. Token có thể được biểu diễn một patch, một hàng pixel hoặc một vị trí trên feature map của CNN. Self-attention giúp mô hình kết hợp thông tin giữa các token, sau đó tổng hợp thành một vector để dự đoán một trong 10 lớp Fashion-MNIST.



Hình 3. Kiến trúc cơ bản của Transformer Classifier

Trong đó, B là số ảnh trong batch, N là số token ảnh, 64 là số chiều biểu diễn của mỗi token.

3.1.2 Tokenizer

Bảng 5. Các cách chia ảnh thành token

Loại	Cách biểu diễn	Đầu ra
patch7	Chia ảnh thành 16 patch kích thước 7×7, chiếu mỗi patch từ 49 lên 64 chiều	`[B,16,64]`
row	Mỗi hàng 28 pixel là một token, chiếu từ 28 lên 64 chiều	`[B,28,64]`
cnn	CNN trích xuất feature map `[B,64,7,7]`; mỗi vị trí không gian thành một token, rồi qua phép chiếu 64 → 64	`[B,49,64]`

3.1.3 Encoder blocks

Transformer classifier bao gồm 2 encoder blocks, dùng để trao đổi thông tin rồi biến đổi đặc trưng, mỗi block gồm hai bước:

$$U = Z + \text{Dropout}\big(\text{MSA}(\text{LayerNorm}(Z))\big)$$
$$Z_{\text{out}} = U + \text{FFN}(\text{LayerNorm}(U))$$

Đây là cấu trúc pre-LayerNorm, tức là chuẩn hóa trước khi đưa vào attention hoặc FFN. Phép cộng với đầu vào là residual connection, giúp giữ thông tin cũ và hỗ trợ truyền gradient.

Hai thành phần có vai trò khác nhau như sau:

- Multi-Head Self-Attention: cho mỗi token tổng hợp thông tin từ các token khác theo trọng số học được. Có 4 head, mỗi head có 16 chiều.
- Feed-forward Network: nhằm biến đổi đặc trưng của từng token qua Linear → GELU → Dropout → Linear → Dropout.

Sau hai block, các token dường như mang thông tin được kết hợp từ toàn bộ chuỗi.

3.1.4 Head phân loại

Áp dụng LayerNorm, rồi lấy token đầu tiên bằng `tokens[:,0]`. Sau quá trình này, vector CLS được dùng làm biểu diễn tổng hợp của ảnh và đưa qua lớp tuyến tính để tạo logits cho 10 lớp.

Tương tự như các head ở E1, forward() thực tế trả về logits và chưa áp dụng softmax vì `nn.CrossEntropyLoss` vốn đã tính sẵn log-softmax.

Khi huấn luyện, logits được đưa vào `CrossEntropyLoss`, khi cần xác suất, `predict_proba()` mới được tính.

3.2 Huấn luyện

Ở vòng huấn luyện này, dùng lại vòng lặp huấn luyện của phần E1, cố định các thiết lập, dùng lại cách chia tập dữ liệu đã nói ở phần đầu tiên:

- Chia tập train/val/test: 48.000/12.000/10.000
- Adam với learning rate là 10^{-3}.
- Batch 128, tối đa 50 epoch, patience là 5, chọn theo val accuracy, khôi phục checkpoint tốt nhất và seed 42.

Với mỗi tokenizer, bản "from-scratch" được huấn luyện trước và trạng thái khởi tạo của nó sẽ được lưu lại. Bản "torch" - hay còn gọi là phiên bản dùng **MultiheadAttention**, nạp đúng trạng thái khởi tạo đó, và dùng cùng thứ tự batch.

3.3 Bảng số liệu và kết quả

Qua huấn luyện, các thông số được kiểm tra cơ bản như sau:

- MSA scratch và torch cho output và attention trùng nhau với `atol=1e-5`, số tham số là 16.640 cho cả hai.
- Khi dùng chung trọng số, $\max|\Delta\text{logits}|$ là $7{,}15 \cdot 10^{-7}$ (patch7), $6{,}41 \cdot 10^{-7}$ (row) và $5{,}51 \cdot 10^{-7}$ (cnn).
- Mỗi tokenizer overfit được một batch 64 ảnh sau 200 bước, với loss cuối lần lượt 0,0094 / 0,0071 / 0,0090 (đều dưới 0,05).

Bảng 6. Kết quả sáu cấu hình được chạy

Mô hình	Token ảnh	Tham số	Best ep / số epoch	s/ep	Latency (ms)	Train (eval)	Val	Test
patch7_scratch	16	72.074	25 / 30	2,3	0,29	91,60%	89,52%	88,01%
patch7_torch	16	72.074	25 / 30	2,3	0,27	91,39%	89,20%	88,04%
row_scratch	28	71.498	36 / 41	3,1	0,29	94,12%	90,64%	89,26%
row_torch	28	71.498	24 / 29	2,8	0,26	92,81%	90,57%	89,47%
cnn_scratch	49	93.962	15 / 20	6,4	0,43	93,97%	91,93%	**91,13%**
cnn_torch	49	93.962	21 / 26	6,3	0,41	94,78%	**92,11%**	91,04%

Qua đó, có thể thấy được, mô hình tốt nhất theo validation là `cnn_scratch` với val accuracy 92,11%. Và ta có thể nhận xét thành hai nhóm:

- **Nhóm 1: so sánh với cùng tokenizer nhưng khác nhau ở scratch và torch:**
  - Về số parameters, số lượng tham số của mô hình scratch và torch ở các tokenizer đều giống nhau, khoảng 72k cho patch7×7, 71k cho row chia thành 28 tokens, hoặc 93k cho CNN.
  - Nhưng có sự khác biệt đôi chút về best epoch, tức là epoch có val accuracy cao nhất, cũng là checkpoint được giữ lại.
  - Với tokenizer là patch 7×7 thì mô hình MSA scratch và torch đều không có sự khác nhau về số lượng tham số, best epoch, không có sự khác nhau quá lớn hoặc đáng kể ở thời gian training, train acc/val acc/test acc.
  - Với tokenizer là row 28px thì hai mô hình có sự khác nhau nhiều hơn và thấy rõ mô hình viết tay scratch sẽ có các giá trị lớn hơn mô hình native, như thông tin best epoch sẽ cho biết tốc độ hội tụ của mô hình native tốt hơn mô hình scratch khoảng 12 epoch, điều này dẫn tới thời gian training của scratch cũng sẽ lớn hơn torch vì early stopping, nếu nhìn vào test acc cuối cùng để đánh giá, thì mô hình torch nhỉnh hơn scratch 0.21 điểm, không quá lớn.
  - Với tokenizer là CNN, tương tự với cách so sánh ở Row, thì mô hình torch có kết quả kém hơn scratch một chút.
- **Nhóm 2: so sánh với cùng loại mô hình nhưng khác tokenizer:**
  - Với scratch, patch 7×7 cho test acc thấp nhất khi so với Row và CNN (**88.01%** < **89.26%** < **91.13%**), tuy best epoch của Row là 36 (cao nhất) và CNN là 15 (nhất nhất) nhưng cả hai mô hình có hai tokenizer này thì có thời gian training gần sát nhau (139s với 138s), điều này còn phản ánh một điều, tuy dừng sớm nhưng CNN với số lượng tham số nhiều hơn hẳn so với Row đồng thời đi qua hai lớp conv nên thời gian training network này chắc chắn phải chậm hơn các mô hình tokenizer khác.
  - Với torch, xét về test acc sau cùng thì CNN vẫn tốt hơn Row sau đó mới đến Patch, best epoch của ba mô hình có tokenizer này cũng có sai khác không lớn, tuy nhiên, với cùng lập luận trên, ở cùng số epoch thì CNN phải tiêu tốn thời gian training nhiều hơn hẳn các mô hình tokenizer còn lại.

Bảng 7. Kiểm định McNemar giữa scratch và torch trên dự đoán test

Tokenizer	Tỉ lệ dự đoán trùng	b	c	p
patch7	93,91%	257	260	0,930
row	92,86%	290	311	0,415
cnn	93,33%	294	285	0,740

Kiểm định **McNemar** lấy nhãn thật và dự đoán của từng model, bỏ qua probs vì kiểm định này chỉ cần biết đúng hay sai.

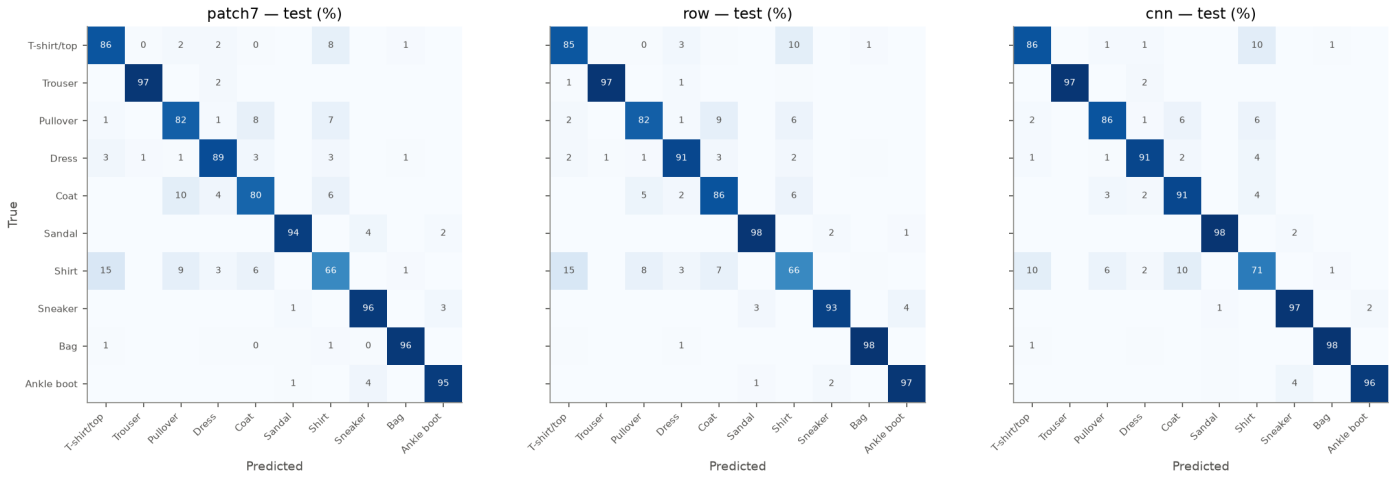
- `b`: số ảnh scratch đoán đúng nhưng torch sai.
- `c`: ngược lại.

- Những ảnh cả hai cùng đoán đúng, hoặc cùng sai thì không mang thông tin so sánh, nên bỏ qua.

Giả thuyết H0: hai model tốt như nhau, khi đó mỗi ảnh "bất đồng" có xác suất 50/50 rơi vào nhóm b hoặc nhóm c, tức là b tuân theo phân phối Binominal(b+c, 0.5)

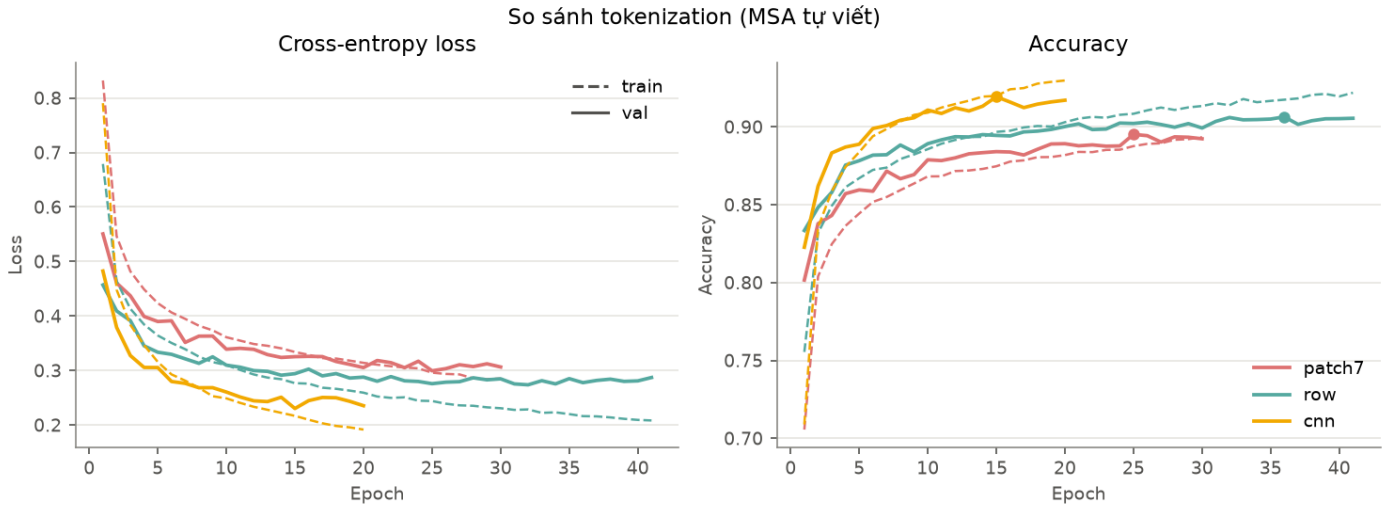
Qua bảng kết quả, `p` cho xác suất thấy độ lệch giữa b và c lớn nếu hai model thật sự ngang nhau. Nếu $p < 0.05$ tức là 5% thì mới tạo ra sự khác biệt thống kê, qua đó, giá trị của p ở các tokenzier khác nhau giữa scratch và torch đều lớn hơn nhiều so với 0.05

→ Không có bằng chứng nào cho thấy MSA scratch và torch dùng `nn.MultiheadAttention` cho chất lượng khác nhau.

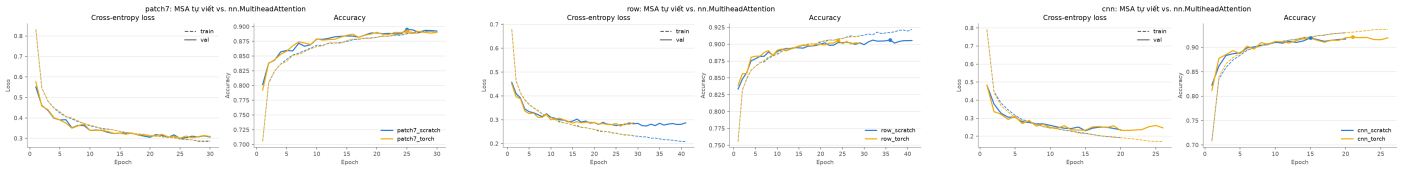


Hình 4. Confusion matrix trên test (%) của ba tokenizer, bản MSA scratch

3.4 Đường cong học



Hình 5. So sánh tokenizer (MSA scratch). Nét đứt là train trong lúc tối ưu (dropout bật), nét liền là val, chấm tròn là checkpoint được chọn.



(a) patch7: scratch vs torch **(b)** row: scratch vs torch **(c)** cnn: scratch vs torch

Hình 6. MSA scratch (xanh) và `nn.MultiheadAttention` (vàng) với từng tokenizer.

- **cnn** hội tụ nhanh nhất: val accuracy vượt 88% từ epoch 3 và đạt đỉnh ở epoch 15 (scratch). Sau đó val loss đi ngang hoặc tăng nhẹ trong khi train loss tiếp tục giảm.
- **row** đạt val accuracy khoảng 90% nhưng val loss gần như đi ngang sau khoảng epoch 20, trong khi train loss vẫn giảm. Bản scratch chạy tới 41 epoch, và gap của nó là 3,5, lớn nhất trong bảng.
- **patch7** có train và val đi sát nhau (gap khoảng 2) và đều cải thiện chậm. Biểu hiện này giống giới hạn về biểu diễn/capacity hơn là overfitting.
- Với mỗi tokenizer, đường val của scratch và torch gần như chồng lên nhau. Kết quả này nhất quán với McNemar ở Bảng 7.

3.5 Nhận xét

Tokenizer ảnh hưởng nhiều hơn bản cài đặt attention. Với cùng depth và D, test accuracy tăng theo thứ tự patch7 (khoảng 88,0%) $<$ row (khoảng 89,3–89,5%) $<$ cnn (khoảng 91,0–91,1%). Ngược lại, scratch và torch khác nhau tối đa 0,32 điểm val, và McNemar không phát hiện khác biệt nào ($p \geq 0{,}415$).

Lợi thế của CNN tập trung ở nhóm áo: Coat 79,5 → 91,0 và Pullover 81,7 → 86,0 (so với patch7), Coat→Pullover giảm còn 3%. Tuy nhiên, Shirt→Coat lại tăng lên 10%. Và cái giá là chi phí cao hơn, khoảng 6,3-6,4 s/epoch so với 2,3s của patch7 và latency CPU cũng cao hơn khoảng 0,15 điểm. Chi phí này đến từ các lớp conv và dùng 50 token thay vì 17.

4 E3

4.1 Kiến trúc

4.1.1 Chuyển ảnh thành chuỗi

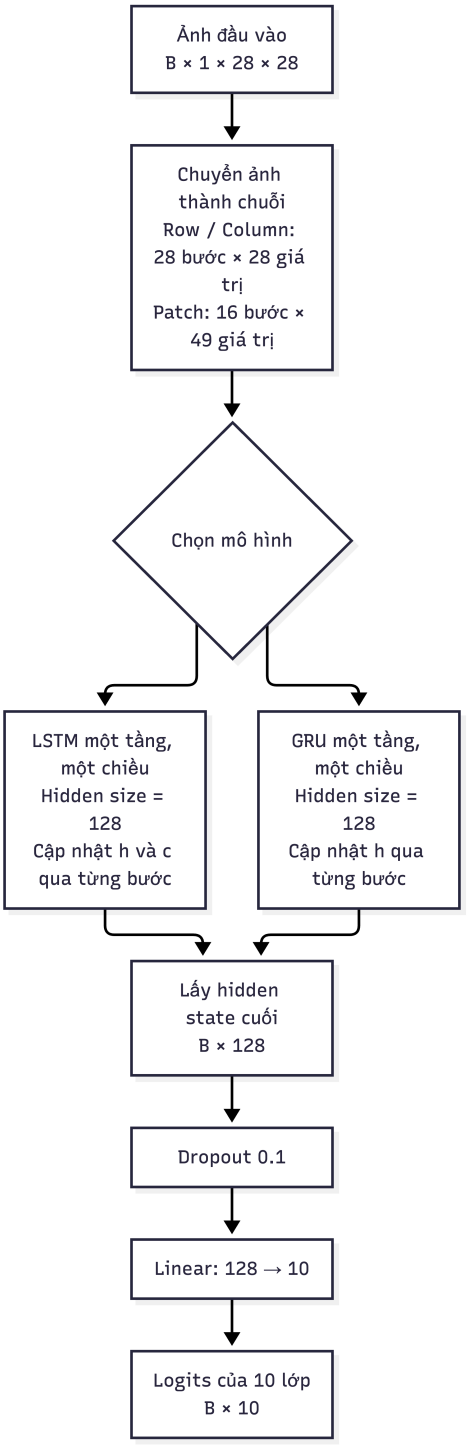
Đâu vào của LSTM hoặc GRU có dạng `[B,T,D]`, mà ảnh ở input đang có dạng `[B,1,28,28]` nên phải thực hiện bước chuyển ảnh thành chuỗi, với `T` là số bước và `D` là số giá trị đầu vào tại mỗi bước.

4.1.1.1 Row-by-row

- Một bước thời gian là một hàng pixel, đọc từ trên xuống dưới.
- Trong đó, `[B, T, D]` có dạng `[B, 28, 28]`
- Xét trên tập dữ liệu Fashion-MNIST: hình dạng quần áo thay đổi rõ theo chiều dọc: như từ cổ áo -> tay áo -> thân...

4.1.1.2 Column-by-column

- Một bước thời gian là một cột pixel, đọc từ trái sang phải.
- Trong đó, `[B, T, D]` có dạng `[B, 28, 28]`
- Xét trên dataset, giống với row nhưng khác hướng quét, đây được xem là thí nghiệm có kiểm soát.



Hình 7. Kiến trúc cơ bản của RNN

4.1.1.3 Patching 7x7

- Một bước thời gian là một patch 7x7 (tương tự như patch 7x7 đã làm ở E2), sẽ chia ảnh thành lưới 4x4 (vì ảnh có kích thước 28x28), đọc từ trái sang phải và từ trên xuống dưới.
- `[B, T, D]` có dạng `[B, 16, 49]`.

4.1.2 LSTM

LSTM (*Long Short-Term Memory*) có hai trạng thái:

- **Cell state c_t:** lưu thông tin tích lũy qua các bước.
- **Hidden state h_t:** biểu diễn đầu ra tại bước hiện tại, đồng thời được truyền sang bước tiếp theo.

Tại bước t, LSTM nhận x_t, h_{t-1} và c_{t-1}. Ba cổng điều khiển cách cập nhật bộ nhớ:

- Forget gate f_t: giữ lại bao nhiêu thông tin từ bộ nhớ cũ
- Input gate i_t: Ghi vào bao nhiêu thông tin mới
- Output gate o_t: Đưa ra bao nhiêu thông tin từ bộ nhớ hiện tại
- Candidate g_t: Nội dung mới có thể được ghi vào bộ nhớ

Các cổng dùng sigmoid nên có giá trị trong khoảng [0, 1]. Candidate dùng `tanh`, bộ nhớ và trạng thái ẩn được cập nhật theo:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$
$$h_t = o_t \odot \tanh(c_t)$$

Trong đó, $\odot$ là phép nhân từng phần tử. **LSTM có ba cổng và một nhánh tạo candidate**.

4.1.3 GRU

GRU (*Gated Recurrent Unit*) đơn giản hơn LSTM: **chỉ duy trì h_t, không có cell state riêng**. GRU có hai cổng:

- **Reset gate r_t:** điều chỉnh mức sử dụng trạng thái cũ khi tạo thông tin ứng viên.
- **Update gate z_t:** cân bằng giữa việc giữ trạng thái cũ và nhận thông tin mới.

Nếu z_t gần 1, mô hình giữ nhiều thông tin cũ; nếu gần 0, mô hình cập nhật nhiều từ candidate n_t.

GRU có **hai cổng và một nhánh candidate**, nên thường cần ít tham số hơn LSTM khi dùng cùng kích thước đầu vào và trạng thái ẩn.

4.2 Huấn luyện

Dùng lại vòng lập huấn luyện của E1 và E2 với cùng cấu hình và thiết lập:

- Adam với learning rate 10^{-3}, batch 128, tối đa 50 epoch, patience 5
- Chọn theo val accuracy, khôi phục checkpoint tốt nhất, seed 42

4.3 Bảng số liệu và kết quả

Bảng 8. Kết quả RNN và baseline huấn luyện lại

Mô hình	*T*	*D*	Tham số	Best ep / số epoch	Time (s)	s/ep	Latency (ms)	Train (eval)	Val	Test	Gap (điểm %)
lstm_row	28	28	82.186	24 / 29	54,3	1,6	0,27	94,43%	**90,71%**	89,82%	3,7
lstm_col	28	28	82.186	22 / 27	43,5	1,4	0,23	93,51%	89,87%	89,01%	3,6
lstm_patch	16	49	92.938	21 / 26	34,3	1,1	0,15	95,28%	89,39%	88,40%	5,9
gru_row	28	28	61.962	14 / 19	65,3	3,1	0,22	92,73%	90,53%	**89,91%**	2,2
gru_col	28	28	61.962	20 / 25	81,6	2,9	0,22	93,79%	90,21%	89,24%	3,6
gru_patch	16	49	70.026	14 / 19	37,7	1,8	0,14	93,93%	89,42%	88,16%	4,5
MLP (retrain)	–	–	535.818	36 / 41	36,6	0,8	0,02	96,57%	90,54%	89,41%	6,0
CNN (retrain)	–	–	421.642	12 / 17	67,5	3,6	0,13	96,57%	93,08%	92,24%	3,5

- LSTM có 4 cổng: input, forget, cell, output. Số tham số là 4·(D·H + H·H + 2H)
- GRU có 3 cổng: reset, update, new. Số tham số là 3·(D·H + H·H + 2H)
- Vì vậy, với cùng H, GRU luôn có đúng 3/4 tham số của LSTM.

So sánh với kết quả ở bảng trên:

- **Số tham số** của GRU ở các cách tokenizer đều bằng 3/4 so với LSTM.

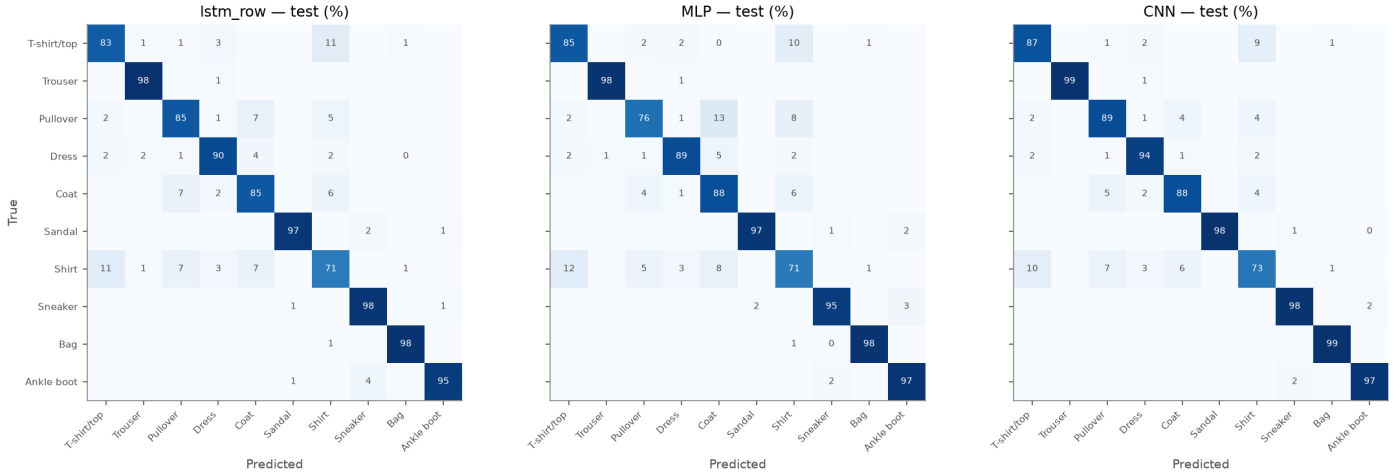
Bảng 9. McNemar trên dự đoán test

A	B	Val A / B	Test A / B	Trùng dự đoán	A đúng, B sai	A sai, B đúng	p
lstm_row	gru_row	90,71 / 90,53	89,82 / 89,91	92,75%	308	317	0,749
lstm_col	gru_col	89,87 / 90,21	89,01 / 89,24	91,70%	351	374	0,414
lstm_patch	gru_patch	89,39 / 89,42	88,40 / 88,16	90,91%	395	371	0,406
lstm_row	MLP	90,71 / 90,54	89,82 / 89,41	91,16%	401	360	0,147
lstm_row	CNN	90,71 / 93,08	89,82 / 92,24	91,43%	263	505	**p < 0,001**

- Model "tốt nhất" được chọn theo val accuracy, trong đó `lstm_row` có acc đạt 90.71%, `gru_row` chỉ đạt 90.53%.
- Dù trên test thì `gru_row` nhỉnh hơn `lstm_row` với điểm số 89.91% so với 89.82%, tuy nhiên ta không được chọn model theo test, làm vậy là dùng tập test để ra quyết định, trong khi con số test báo cáo sẽ bị lạc quan.

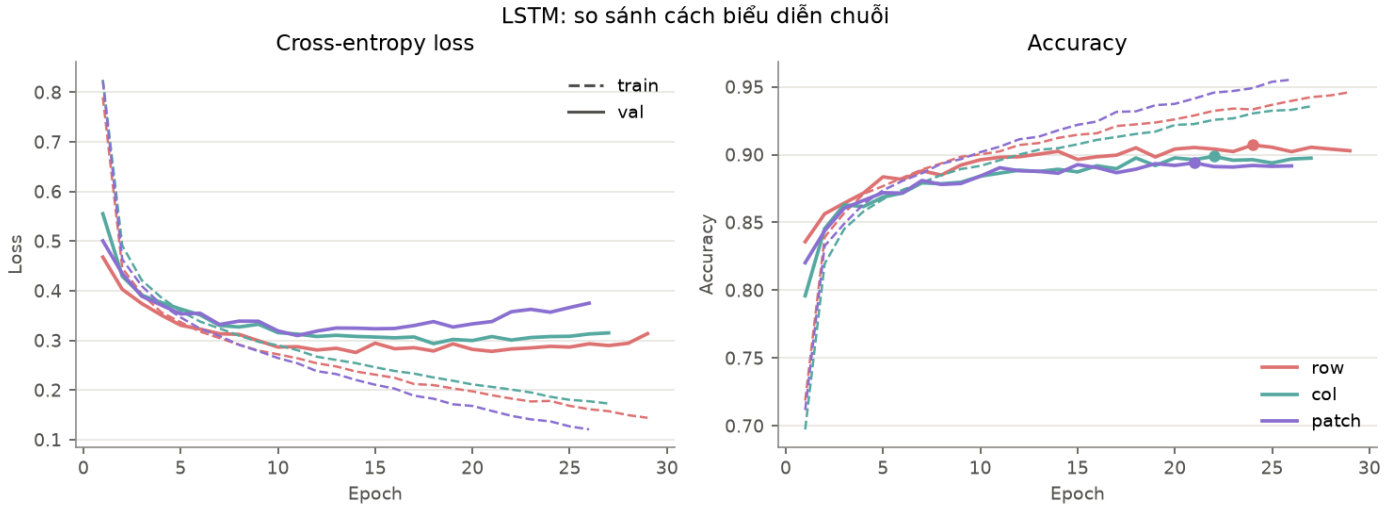
Do đó:

- LSTM và GRU: có $p > 0.4$ ở cả ba cách biểu diễn, lớn hơn nhiều so với 0.05 (5%), tức là không có khác biệt có ý nghĩa thống kê.
- RNN tốt nhất (`lstm_row`) và MLP: có $p \approx 0.15$, chênh lệch 0.17 điểm, chưa đủ kết luận để nói rằng RNN tốt hơn.
- RNN tốt nhất (`lstm_row`) và CNN: có $p < 0.001$, CNN đúng ở 505 ảnh mà RNN sai, trong khi ngược lại RNN chỉ đúng khoảng 263, tức là tỉ lệ CNN đúng gấp đôi RNN đúng. Điều này cho thấy CNN tốt hơn là có ý nghĩa thống kê.

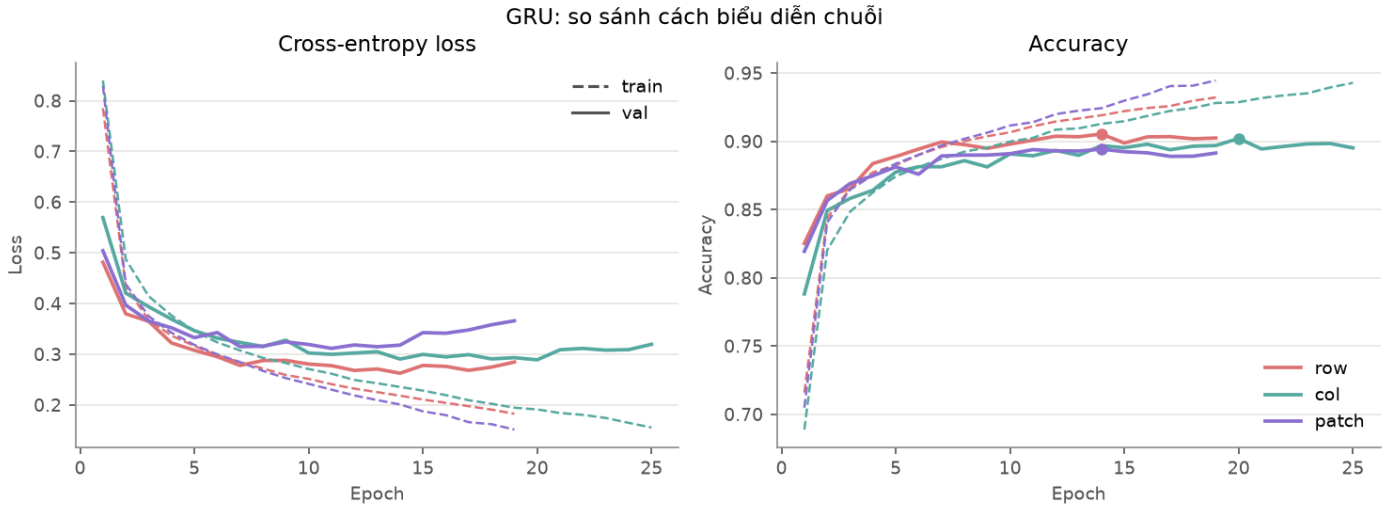


Hình 8. Confusion matrix trên test (%) của RNN tốt nhất theo val (`lstm_row`) và hai baseline huấn luyện lại trong E3

4.4 Đường cong học

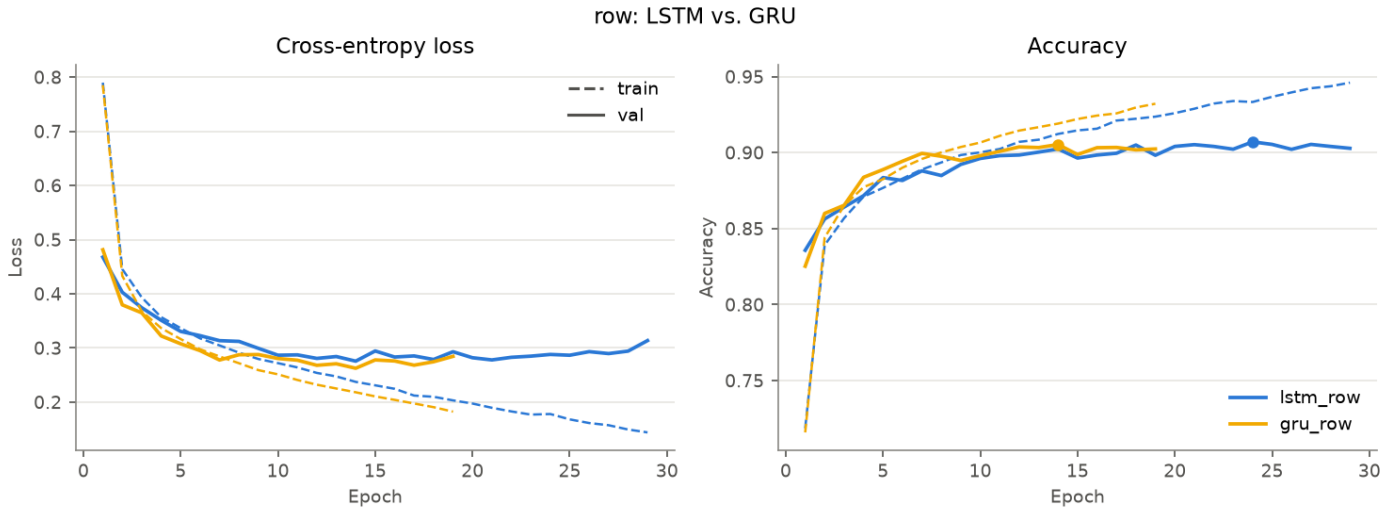


(a) LSTM: so sánh row, col, patch

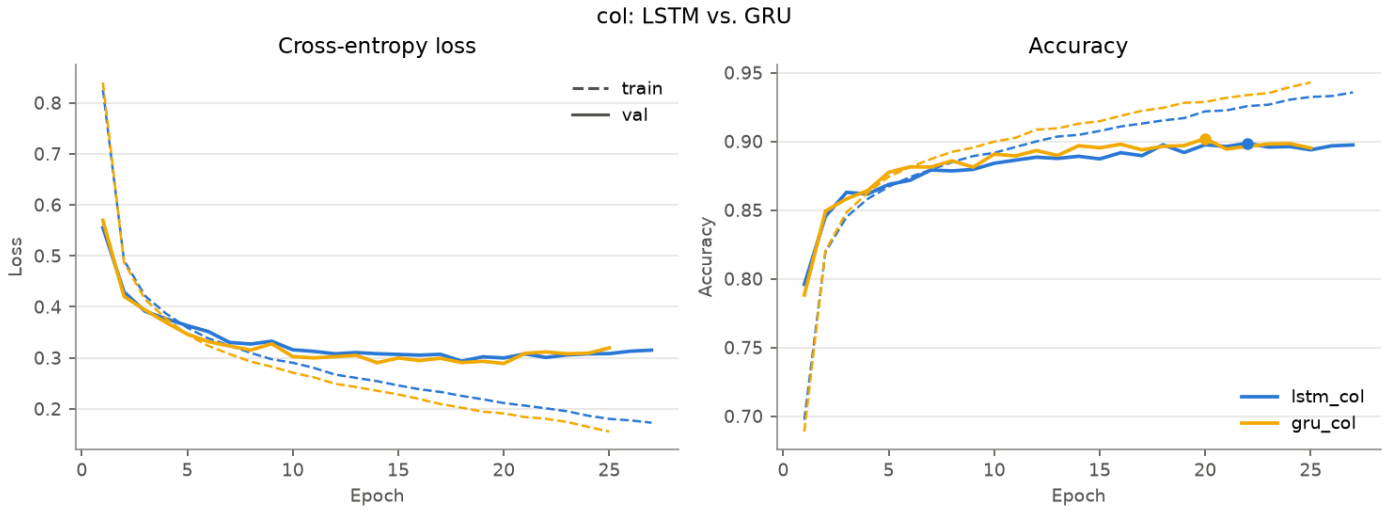


(b) GRU: so sánh row, col, patch

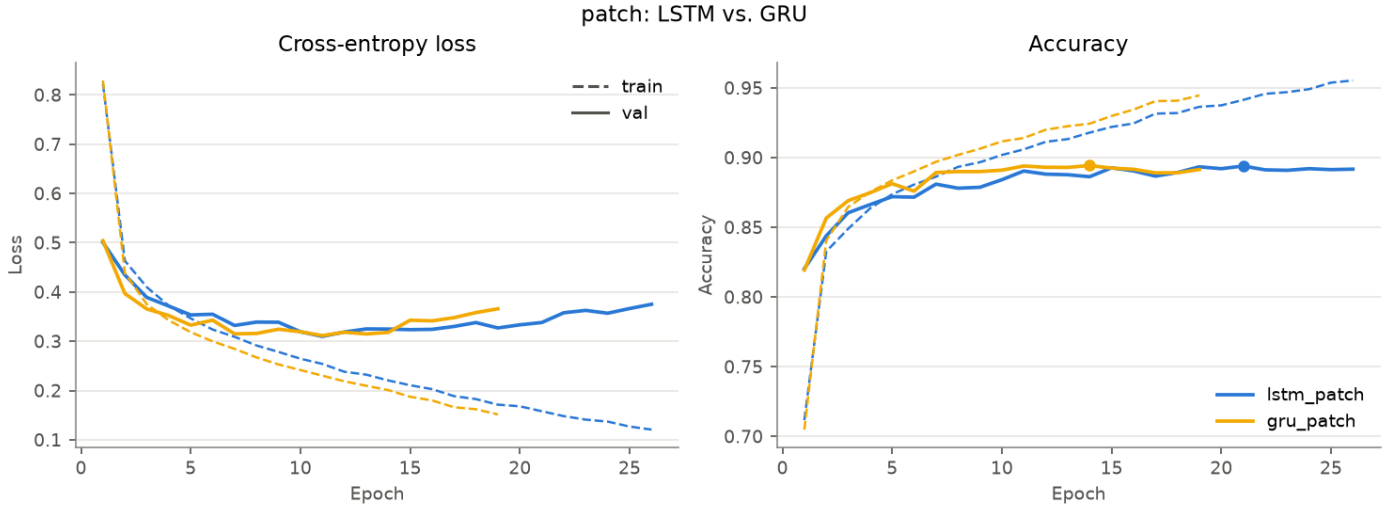
Hình 9. So sánh cách biểu diễn chuỗi với LSTM và với GRU. Nét đứt là train trong lúc tối ưu, nét liền là val, chấm tròn là checkpoint được chọn.



(a) row: LSTM vs GRU



(b) col: LSTM vs GRU



(c) patch: LSTM vs GRU

Hình 10. LSTM (xanh) và GRU (vàng) với cùng một cách biểu diễn

- **patch** overfit rõ nhất. Với LSTM, val loss tăng từ khoảng epoch 10–12 (lên khoảng 0,37 ở epoch cuối), trong khi train loss giảm xuống khoảng 0,12. Gap của lstm_patch là 5,9, lớn nhất trong nhóm RNN.
- **row** có val loss thấp và ổn định nhất (khoảng 0,28). Val accuracy của row cao hơn col và patch ở hầu hết các epoch.
- GRU dừng sớm hơn LSTM với row và patch (19 so với 29 và 26 epoch). Đường val của hai loại cell đan xen nhau, không bên nào vượt trội rõ rệt.

4.5 Nhận xét

- **Theo cách biểu diễn:**
  - row tốt nhất trong ba cách biến ảnh thành chuỗi (row | col | patch), với test_acc là 89.8%∼89.9%, col kém hơn khoảng 0.7 điểm, patch thì kém hơn khoảng 1.4∼1.7 điểm so với row. Thứ tự này giống nhau ở cả LSTM và GRU.
  - row và col có cùng T, D và số tham số tham gia mô hình, nên chênh lệch này giữa chúng đến từ hướng quét. Đọc ảnh từ tren xuống dưới (row) có lợi hơn đọc từ trái sang phải, vì hình dạng quần áo thay đổi theo chiều dọc rõ hơn.
- **So với Baseline:**
  - LSTM và GRU có accuracy ngang với MLP nhưng dùng ít tham số hơn rất nhiều.
  - Tất cả RNN đều kém hơn CNN khoảng 2.4, 2.5 điểm trở lên đến khoảng 4.5 điểm.
- **Thời gian train:**
  - Patch có thời gian train nhanh nhất trong số tất cả, vì chỉ có T=16 bước tuần từ, so với T=28 của row/col.
  - Xét về độ trễ (latency), các mô hình RNN rơi vào khoảng 0.14∼0.27ms, gần tương đương với CNN 0.13ms, trong khi đó MLP nhanh nhất chỉ với 0.02ms vì chỉ là vài phép nhân ma trận.
- **Gap:**
  - Gap là khoảng cách giữa train và val.
  - Trong đó, patch có gap lớn nhất khoảng 5.9 điểm, nên patch overfit nhiều nhất.
  - GRU row có gap nhỏ nhất là 2.2
  - MLP có gap = 6.0, do nó sử dụng rất nhiều tham số.